

Lecture 6

Object-Oriented Design Principles

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science

College of Engineering

University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Motivating Examples

Suppose you have this interface...

```
class GoFundMeCampaign(Protocol):  
    def donate(self, credit_card_info: CreditCardInfo,  
               amount: int):  
        ...  
  
    def withdraw(self, amount: int):  
        ...
```

Motivating Examples

...and two interfaces that use that interface.

```
class Donor(Protocol):  
    def donate_to(self, campaign: GoFundMeCampaign, amount: int):  
        ...  
  
class CampaignStarter(Protocol):  
    def __init__(self, campaign: GoFundMeCampaign):  
        ...  
    def withdraw_funds(self, amount: int):  
        ...
```

Does this look fine to you?

Motivating Examples

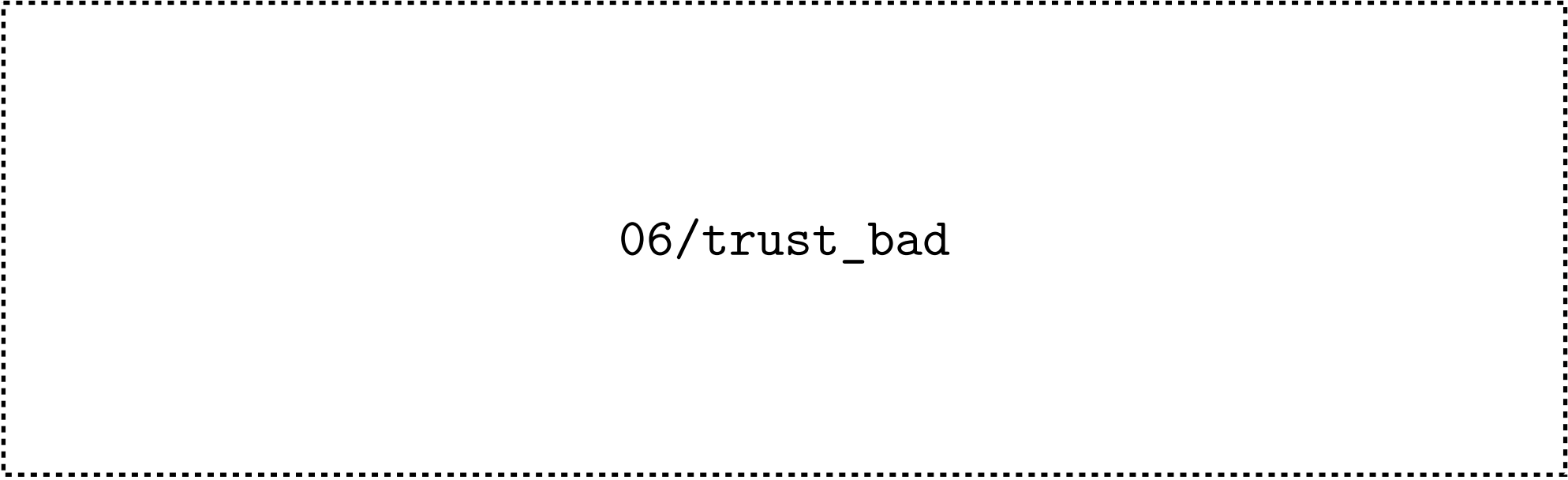
```
class CombinationLockVault:
    def __init__(self, initial_passcode: str):

class TimeLockVault:
    def __init__(self, hours: int):

class Bank:
    def __init__(self):
        self.vault = CombinationLockVault("00000")
```

Does this look fine to you?

Motivating Examples



06/trust_bad

Why is it problematic to introduce Copycat in the same manner?

SOLID Principles

- **SOLID principles:** Set of principles to guide object-oriented design

Principle	Acronym	Summary
Single Responsibility Principle	SRP	Separation of concerns/responsibilities
Open-Closed Principle	OCP	Open for extension, closed for modification
Liskov Substitution Principle	LSP	Behavioral subtyping (Lecture 8)
Interface Segregation Principle	ISP	Interfaces must contain only necessary parts
Dependency Inversion Principle	DIP	Depend on abstractions, not concretions

Single Responsibility Principle (SRP)

- **SRP:** Class should have only one reason to change
- This is *separation of concerns*
 - “Gather together things that change for the same reason; separate those that don’t”

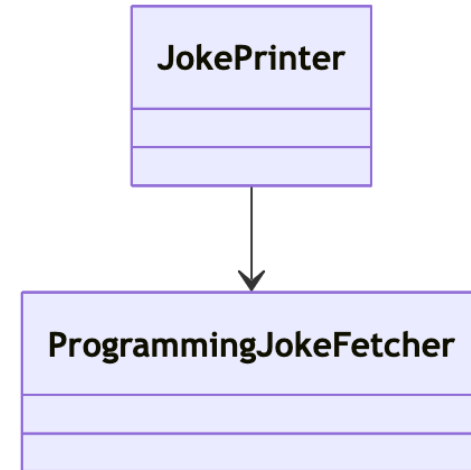
06/joke_printer_bad

06/joke_printer_less_bad

Dependency Inversion Principle (DIP)

- **DIP:** High-level concretions must not depend on low-level concretions; they must both depend on abstractions
 - Informally, “Depend on *abstractions*, not concretions”

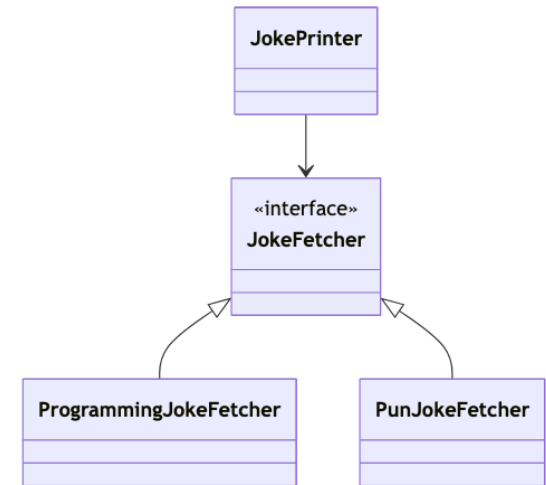
06/joke_printer_less_bad



Dependency Inversion Principle (DIP)

- **DIP:** High-level concretions must not depend on low-level concretions; they must both depend on abstractions
 - Informally, “Depend on *abstractions*, not concretions”

06/joke_printer_good



Interface Segregation Principle (ISP)

Consider a vending machine.



Interface Segregation Principle (ISP)

A vending machine can be interacted with in several ways:

- Enter money
- Choose drink
- Restock cups
- Restock drink
- Change price
- Factory reset

Should all of these be available to Customers?

Interface Segregation Principle (ISP)

- **ISP:** Clients should not be forced to depend upon interfaces that they do not use
 - Break large interfaces into smaller interfaces
 - Smaller interfaces should only expose parts relevant to particular clients

Interface Segregation Principle (ISP)

- VendingMachine class
 - enter_money
 - choose_drink
 - restock_cups
 - restock_drink
 - change_price
 - factory_reset
- Customer class:
 - buy_drink(v: VendingMachine)
- RestockingStaff class:
 - restock_all(v: VendingMachine)
- Why does this violate ISP?
- How do we fix this so it adheres to ISP?

Interface Segregation Principle (ISP)

- VendingMachine class
 - enter_money
 - choose_drink
 - restock_cups
 - restock_drink
 - change_price
 - factory_reset
- DrinkBuyingInterface class:
 - enter_money
 - choose_drink
- RestockingInterface class:
 - restock_cups
 - restock_drink

Make interfaces expose only what is necessary for each client

Interface Segregation Principle (ISP)

```
VendingMachine <: DrinkBuyingInterface  
VendingMachine <: RestockingInterface
```

- Customer class:
 - `buy_drink(v: DrinkBuyingInterface)`
- RestockingStaff class:
 - `restock_all(v: RestockingInterface)`

Interface Segregation Principle (ISP)

More practical example: passing data to the view (MVC)

```
class Player:
    def __init__(self):
        ...

    @property
    def name(self) -> str:
        ...

    def add_points(self, points: int):
        ...
```

```
class PlayerInfo(Protocol):
    @property
    def name(self) -> str:
        ...
```

Interface Segregation Principle (ISP)

- **ISP violation:** View can invoke non-presentation behavior:

```
class View:  
    def show_player(player: Player):  
        ...
```

- **Adheres to ISP:** View can access only immutable data:

```
class View:  
    def show_player(player: PlayerInfo):  
        ...
```

Open-Closed Principle (OCP)

- **OCP:** Software entities should be *open for extension*, but *closed for modification*
 - “Add new code instead of modifying existing code”
 - Reduces chance of breaking existing functionality
- For OOP: Classes ideally are not modified after being defined
 - Use *subtype polymorphism* to support new features
- If not possible for existing design:
 - Refactor existing code (ideally guided by unit tests)
 - Add new code to support new feature

Open-Closed Principle (OCP)

Game scenario: Checking whether a unit can attack another in a grid-based game



Open-Closed Principle (OCP)

Units have differing valid attack patterns:

- Soldier: Exactly 1 cell, orthogonal only
- Fencer: 1-2 cells (inclusive), orthogonal only
- Archer: 1-5 (inclusive) Manhattan distance range

```
class AttackValidator:  
    def can_attack(self, unit_type: UnitType,  
                  src: Location,  
                  target: Location) -> bool:
```

What is the easiest way to implement can_attack?

Open-Closed Principle (OCP)

06/attack_validator_bad

- `can_attack` violates OCP for this case; it cannot support new unit types without being modified
 - Add new `UnitType.GUNNER` with 8-cell Manhattan distance (min 1) valid attack pattern
- How do we refactor the code in a way that adheres to OCP for this case?

Open-Closed Principle (OCP)

```
06/attack_validator_good
```

How do we support the new `UnitType.GUNNER` (min 1, max 8 Manhattan) without modifying existing code?

Open-Closed Principle (OCP)

Simply add a new AttackValidator subclass for the Gunner unit type:

```
# GunnerAttackValidator <: AttackValidator
class GunnerAttackValidator:
    def can_attack(self, src: Location, target: Location) -> bool:
        return 1 <= src.manhattan(target) <= 8
```

New features can be introduced *without editing existing code*

- “Make the *change easy* (which might be hard), then make the *easy change*” ([source](#))

Warning on applying principles

Do **not** blindly apply SOLID principles everywhere!

- Extra effort needed to keep code flexible
- More abstractions $\Rightarrow$ harder to understand code

Is it worth the upfront cost to make future changes easier?

- Does the code need to be flexible?
- How urgent is the change needed?
- How likely will this aspect of the code change?
- How drastic will future changes likely be?

Tip

Practicality beats purity (most of the time).